

What does an AI actually mean by "remembering"?

Does it keep a perfect recording of everything you said — or let the past quietly reshape it?

CHAPTER 1

The Two Paths of Memory

A 50-turn conversation. Turn 3 sets a precise financial limit; Turn 50 asks for it back. How should an AI remember?

TURN 3 · CLIENT PROPOSAL

"Section 14.2: The maximum indemnity cap is set to \$1,450,000 with net 30 payment terms."

----- 46 turns of intervening discussion -----

TURN 50 · QUERY

"What was the exact financial indemnity cap we agreed on earlier?"

How should the model respond?

STRATEGY A Verbatim Store

"In Section 14.2, the maximum indemnity cap is set to \$1,450,000 with net 30 payment terms."

Recall Style: 100% Exact
Memory: Stores all past tokens

STRATEGY B Compressed State

"The agreement specifies an indemnity limit of approximately \$1.45M with standard 30-day payment settlement terms."

Recall Style: Semantic Gist
Memory: Fixed-size internal state

Teaching simplification — illustrates exact retrieval vs. semantic compression.

Which strategy would you deploy?

Scenario 1: Legal Contract Audit & Compliance

Deploy Verbatim Store

Deploy Compressed State

Scenario 2: Real-Time Voice Chat Assistant

Deploy Verbatim Store

Deploy Compressed State

Why does this tradeoff exist? To understand, we must inspect what a Transformer computes and caches during generation.

CHAPTER 2

Naive Recompute vs. The Cache

Autoregressive generation predicts one token at a time. Without caching, the model recalculates Key and Value projections for all past tokens at every step ($O(N^2D^2)$ work). KV caching stores these projections once,

cutting projection work to $O(ND^2)$.

Sequence Length (N): 80 tokens

D = 32 (Hidden Dimension)

Float64 TypedArray

NO CACHE — RUNTIME

24.25 ms

Recomputing past K/V projections

KV CACHE — RUNTIME

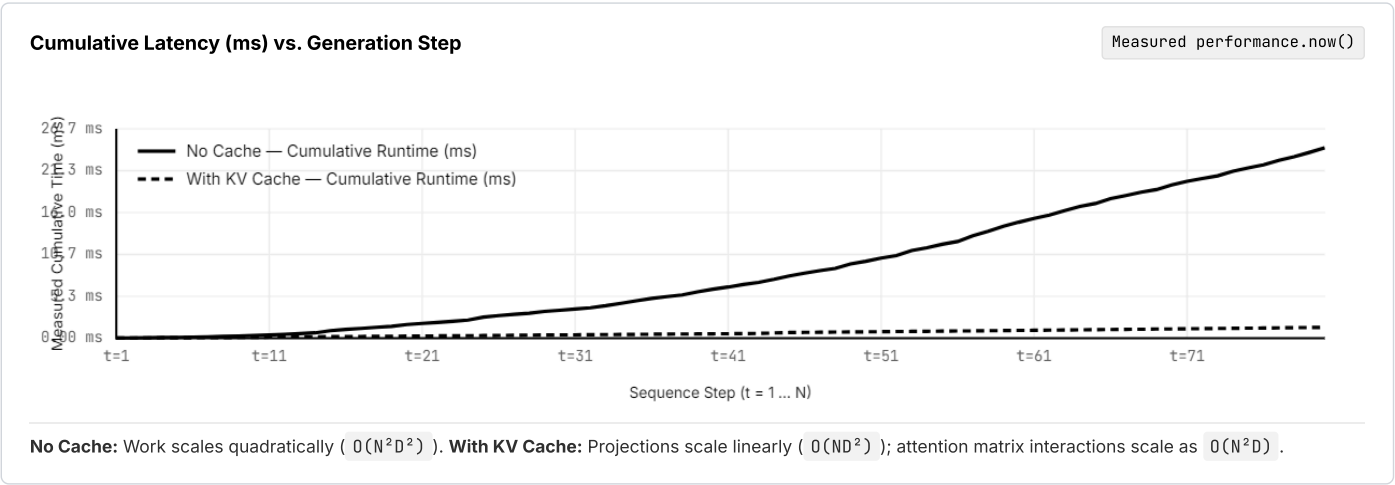
1.38 ms

Single-pass K/V projection reuse

MEASURED SPEEDUP

17.6×

Wall-clock latency ratio



OPERATION STAGE	STEP T FORMULA	CUMULATIVE WORK (N TOKENS)	COMPLEXITY
Naive K/V Projection (No Cache)	$4 \cdot t \cdot D^2$	$2N(N + 1)D^2 = 13,271,040$ FLOPs	$O(N^2D^2)$
Cached K/V Projection (KV Cache)	$4 \cdot D^2$	$4ND^2 = 327,680$ FLOPs	$O(ND^2)$
Attention Interaction ($QK^T + AV$)	$4 \cdot t \cdot D$	$2N(N + 1)D = 414,720$ FLOPs	$O(N^2D)$
Naive Total	$4tD^2 + 4tD$	$2N(N + 1)(D^2 + D) = 13,685,760$ FLOPs	$O(N^2D^2)$
Cached Total	$4D^2 + 4tD$	$4ND^2 + 2N(N + 1)D = 742,400$ FLOPs	$O(ND^2 + N^2D)$

Scope: Accounts for W_K, W_V projections and attention score/value operations.

3D Attention Microscope

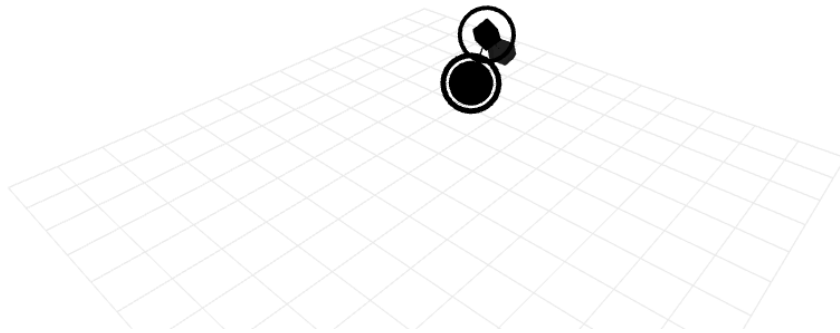
Mechanistic inspection of high-dimensional attention states (D=32) projected deterministically into 3D space.

Step (t): 80 / 80

Cached Tokens: 80

Stored Vectors: 160

Status: Historical K/V reused: 79 · New: 1



Token Inspector

Token k_1

- 1 Dot Product
 $\mathbf{q}_t \cdot \mathbf{k}_i$
-0.793
- 2 Scaled Logit
 $(\mathbf{q}_t \cdot \mathbf{k}_i) / \sqrt{D}$
-0.140
- 3 Attention Weight
 $\mathbf{a}_i^{(t)} = \text{softmax}(\mathbf{S})_i$
100.00% ($\alpha = 1.0000$)
- 4 Value Contribution
 $\mathbf{a}_i^{(t)} \cdot \mathbf{v}_i$
Weighted by α_1 (100.00%) in $\mathbb{R}^{32}$
- 5 Attention Output
 $\mathbf{o}_t = \sum \mathbf{a}_i^{(t)} \mathbf{v}_i$
Projected 3D: [-2.0, -3.4, -7.0]

Query → Similarity → Logit → Softmax → Value weighting

3D Scene: Deterministic projection of actual 32D attention vectors ($\mathbf{P} \in \mathbb{R}^{32 \times 3}$).

Attention Links: Line opacity & width scale directly with attention weight α_i .

• Vaswani et al. 2017 • Multi-Head Attention Formulation

$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q}\mathbf{K}^\top / \sqrt{d})\mathbf{V}$. KV caching eliminates redundant recomputation of past key and value vectors.

Caching saves compute—but every token you cache demands memory. Let's see how fast it expands.

CHAPTER 3

The Cost: What You're Actually Storing

Every token retains Key and Value activations in GPU memory. As context length and batch size scale, KV cache footprint rapidly rivals model parameter weights.

Architectural Evolution: Reducing KV Footprint

MHA · Multi-Head Attention

$n_{KV} = n_Q$

Independent K/V head per query head. Maximum memory footprint (e.g. GPT-3 175B with 96 KV heads).

GQA · Grouped-Query Attention

$n_{KV} < n_Q$

Multiple query heads share each KV head (e.g. Llama-3 with 32 Q heads sharing 8 KV heads). Cuts cache size 4x to 8x.

MLA · Multi-Head Latent Attention

$c_t^{KV} \in \mathbb{R}^{512} + k_t^R \in \mathbb{R}^{64}$

Compresses KV into a shared 512D latent vector + 64D decoupled RoPE key (576 elements total per layer).

Governing KV Cache Formulas

Conventional MHA / GQA

Pope et al. 2023

$VRAM_{KV} = 2 \times L \times n_{KV} \times d_{head} \times T \times B \times s$

Factor of 2 accounts for separate Key and Value tensors.

DeepSeek-V2 MLA

DeepSeek-AI 2024

$VRAM_{MLA} = L \times (d_c + d_r) \times T \times B \times s$

Low-rank latent $d_c = 512$ + RoPE $d_r = 64$ (576 elements/layer). No factor of 2.

Interactive VRAM Sandbox

GiB (1 GiB = 2³⁰ bytes)

Model Architecture Preset

Llama-3-8B — GQA: 8 KV heads

GQA (8 KV heads, 4:1 ratio) ·
14.96 GiB weights (FP16 / BF16
(16-bit))

Context Length (T)

8,192 tokens

Batch Size (B)

1

Precision

FP16 (2 B)

FP8 (1 B)

INT4 (0.5 B)

Target GPU

NVIDIA A100 (80 GB)

Estimated VRAM Allocation

15.96 GiB / 80 GiB (NVIDIA A100 (80 GB))

MODEL WEIGHTS

14.96 GiB

FP16 / BF16 (16-bit) (2 B/elem)

KV CACHE FOOTPRINT

1.00 GiB

4096 B/token/layer · 1,073,741,824 B

TOTAL VRAM

15.96 GiB

20.0% of 80 GiB GPU ceiling

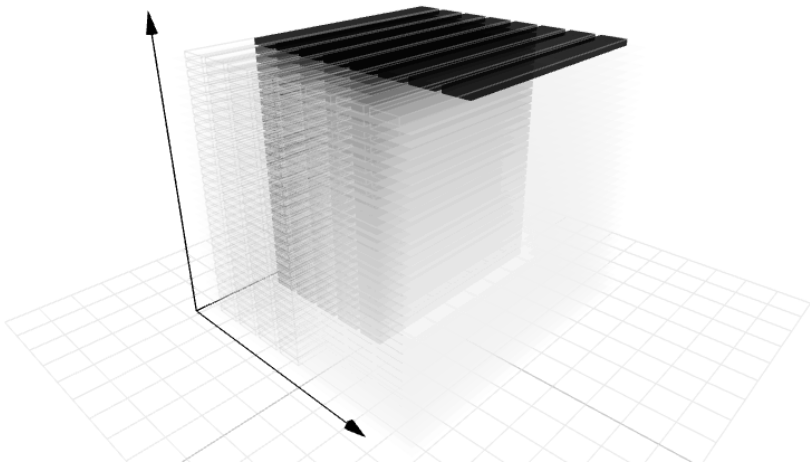
FORMULA EVALUATED

Conventional MHA/GQA (Pope et al. 2023): $M_{KV} = 2 \cdot L \cdot n_{KV} \cdot V \cdot d_{head} \cdot T \cdot B \cdot s \rightarrow 2 \times 32 \times 8 \times 128 \times 8192 \times 1 \times 2 \text{ bytes}$

3D Structural Tensor Dimensions

Layers (L) × KV Channels (n_{KV}) × Tokens (T)

- Z: Context Length (T)
- Y: Decoder Layers (L)
- X: KV Heads / Latents



Visualizes the structural tensor (L × n_{KV} × T). VRAM accounts for head width, batch size, and precision.

This growing buffer isn't just a storage problem—it fundamentally alters how fast the GPU can generate tokens.

Limitations in Production: Prefill vs. Decode

Inference splits into two opposite hardware regimes: parallel compute-bound prefill vs. memory-bandwidth-bound decode.

Phase 1: Prefill

COMPUTE-BOUND

Matrix-Matrix (GEMM) · Highly Parallel
Processes the entire prompt simultaneously across thousands of GPU Tensor Cores.
Execution: All tokens in a single forward pass.
Intensity: High ($I \gg 150$ FLOP/byte).
Bottleneck: Compute execution units; memory bus is not saturated.

Phase 2: Decode

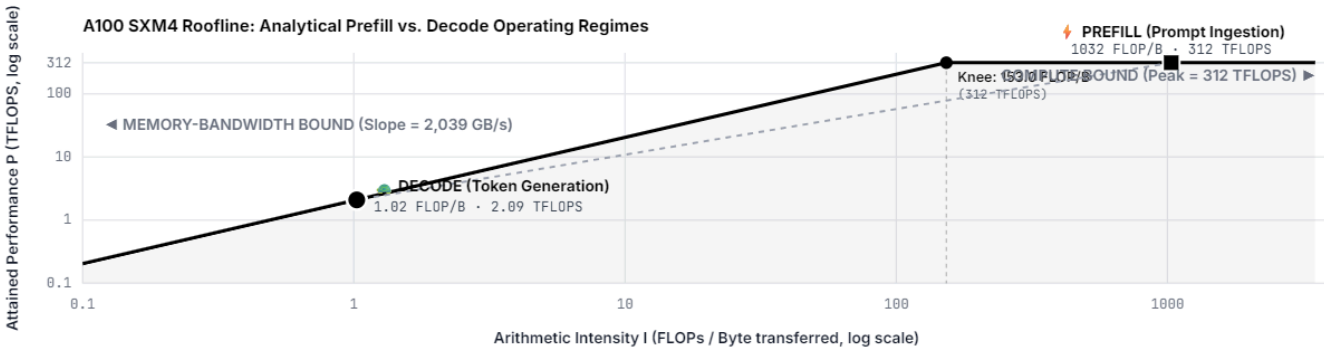
MEMORY-BOUND

Matrix-Vector (GEMV) · Sequential
Generates 1 token at a time, streaming the entire accumulated KV cache from HBM for each step.
Execution: Autoregressive loop, 1 token per sequence.
Intensity: Extremely low ($I \approx 0.99 - 4$ FLOP/byte).
Bottleneck: Memory bandwidth; Tensor Cores wait for HBM data transfers.

Roofline Model: Hardware Ceilings

$$P = \min(P_{\text{peak}}, I \times B_{\text{mem}})$$

NVIDIA A100 80GB SXM · 312 TFLOPS Dense FP16 · 2,039 GB/s HBM2e



- Williams, Waterman, Patterson (2009) · RoofLine Model
- Knee point at $I^* = P_{\text{peak}} / B_{\text{mem}} \approx 153.0$ FLOP/byte separates memory-bandwidth from compute bounds.

Interactive Serving Sandbox: Memory Bandwidth Saturation

Adjust concurrent requests and context length to observe how decode memory traffic limits throughput.

Concurrent Requests / Batch (B)

1 request

Average Context Length (T)

1,024 tokens

1 32 64 96 128 512 4k 8k 16k 32k

MEMORY TRAFFIC / STEP
16.19 GB
Weights: 16.06 GB · KV:
0.13 GB

ARITHMETIC INTENSITY
1.02 FLOP/B
FLOPs / Bytes transferred

ATTAINED COMPUTE
2.09 TFLOPS
0.7% of 312 TFLOPS
dense peak

STEP LATENCY
7.9 ms
Time per token

PER-STREAM RATE
125.9 tok/s
 $1 / t_{\text{step}}$

AGGREGATE THROUGHPUT
126 tok/s
Bandwidth limit ceiling:
~15,192 tok/s

Systems Solutions: PagedAttention & Disaggregation

vLLM (Kwon et al. 2023): PagedAttention eliminates internal memory fragmentation using virtual memory paging.
DistServe (Zhong et al. 2024): Physically disaggregates compute-bound prefill from bandwidth-bound decode onto separate GPU clusters.

Given these physical costs, how do alternative architectures avoid linear KV memory growth?

Alternate Memory Approaches, Compared Honestly

Normalized comparison under a 32-layer, FP16 scale (L=32, d_{model}=4096). Reducing memory footprint always trades off addressability, recall, or precision.

Taxonomy of State Semantics

01 · Explicit Token

Addressable Buffers

Standard KV, StreamingLLM: retains explicit activations.

02 · Compressed Token

Addressable Latents

DeepSeek MLA: low-rank vectors indexed per token.

03 · Associative

Matrix Fast Weights

Linear Attention: folds outer products into a fixed matrix.

04 · Dynamical

Recurrent State

Mamba SSM: continuous hidden state evolving selectively.

05 · Learned Graph

Synaptic Plasticity

Dragon Hatchling (BDH): Hebbian reweighting on fixed topology.

Standard KV-Cache

0(T) Linear

Explicit Token Memory

$$M_{KV}(T) = 2 \cdot L \cdot n_{KV} \cdot d_{head} \cdot s \cdot T = 131,072 \cdot T \text{ B}$$

Retained: Full uncompressed Key and Value projections.

Limitation: Linear memory growth; decode memory-bandwidth bound.

Vaswani 2017

Approach A

Approach B

Attention Sinks

0(1) Window

Windowed Token Memory

$$M(T) = 131,072 \cdot \min(T, 1028) \text{ B } [W_{\text{sink}}=4, W_{\text{win}}=1024]$$

Retained: Initial sink tokens + sliding recent window.

Limitation: Zero exact recall for evicted intermediate tokens.

Xiao 2023
(StreamingLLM)

Approach A

Approach B

Low-Rank MLA

0(T) Compressed

Compressed Token Memory

$$M_{MLA}(T) = L \cdot (d_c + d_r) \cdot s \cdot T = 36,864 \cdot T \text{ B}$$

Retained: 512D latent + 64D decoupled RoPE key.

Limitation: 71.9% smaller footprint, but remains O(T) linear.

DeepSeek-AI 2024

Approach A

Approach B

Linear Attention

0(1) Matrix

Associative Matrix

$$M_{\text{Linear}} = L \cdot n_{\text{head}} \cdot r \cdot d_v \cdot s = 33.55 \text{ MB}$$

Retained: Associative matrix accumulating outer products.

Limitation: Lacks exact softmax focus; finite state capacity.

Katharopoulos 2020

Approach A

Approach B

Mamba (SSM)

0(1) Vector

Dynamical State

$$M_{SSM} = L \cdot d_{\text{model}} \cdot d_{\text{state}} \cdot s = 4.19 \text{ MB}$$

Retained: Continuous-time hidden state evolving selectively.

Limitation: Fixed state complicates exact long-range recall.

Gu & Dao 2023

Approach A

Approach B

Dragon Hatchling

0(1) Graph

Synaptic Plasticity

Fixed synaptic state; length-independent footprint [Ch. 6]

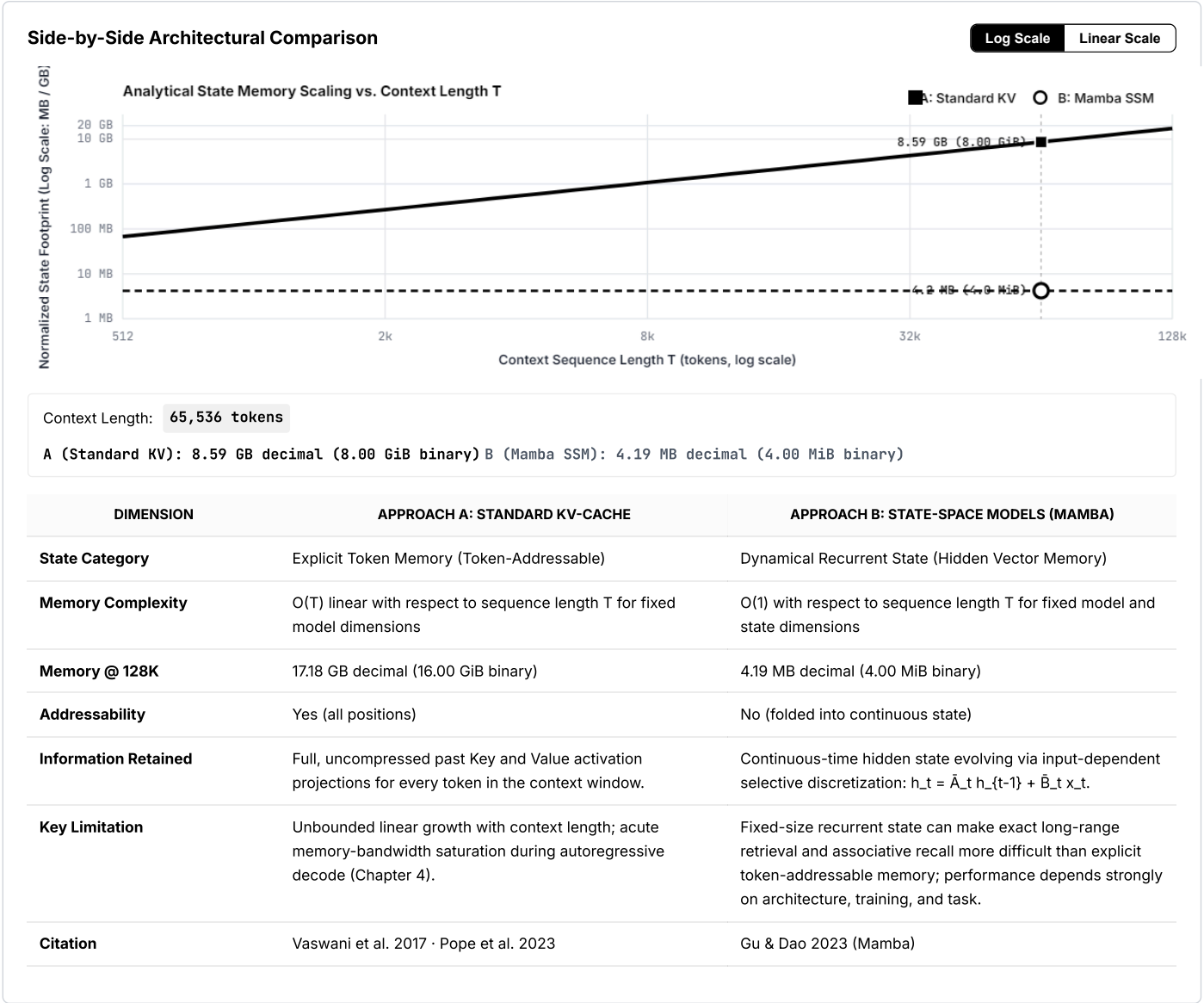
Retained: Synaptic edge connections in a learned bipartite network.

Limitation: Finite capacity; concepts share synaptic pathways.

Kosowski et al. 2025

Approach A

Approach B



Can memory be neither a growing token buffer nor a conventional vector, but an integrated architecture of synaptic plasticity?

BDH as an Integrated Redesign, Not a Tweak

The Dragon Hatchling (arXiv:2509.26507) replaces explicit token buffers with an evolving internal synaptic state, trading exact historical recall for constant memory ($O(1)$ in T).

EVIDENCE FRAMEWORK: [PRIMARY SOURCE] Author paper / code [DERIVED] Mathematical consequence [INTERPRETATION] Pedagogical model

AUTHOR CLARIFICATION · HUGGING FACE PAPER DISCUSSION [PRIMARY SOURCE]

"There are about 5 main differences, which should not be tried in isolation - or they will indeed not work (and cannot work in isolation, as explained in the theory part of the paper)."

Adrian Kosowski (CSO at Pathway) · Oct 7, 2025 [Hugging Face Discussion ↗](#)

Architectural Distinctions: BDH vs. Standard Transformer			
Primary source: Kosowski et al. (arXiv:2509.26507).			
DIMENSION	BDH (DRAGON HATCHLING)	STANDARD TRANSFORMER	STATUS
Attention / State	Recurrent associative state Associative recurrent state updated from current activations.	Softmax attention over tokens $A_t = \text{softmax}(Q_t K_{1:t}^T) V_{1:t}$ over stored tokens.	[PRIMARY SOURCE]
Representation	High-dimensional sparse positive Neuron activations $a_t \geq 0$ in high dimension N .	Dense continuous activations Continuous dense vectors in moderate dimension.	[PRIMARY SOURCE]
Key-Query Match	Sparse-overlap interpretation Associative retrieval via overlapping sparse support.	Dot-product similarity Scaled dot products normalized by softmax.	[INTERPRETATION]
State Scaling	Fixed O(1) state w.r.t. T State footprint does not grow with context.	Linear O(T) growth Allocates one persistent KV entry per token.	[DERIVED]
Memory Mechanism	Synaptic plasticity Working memory is written into evolving synaptic connections S_t .	Explicit KV cache buffer External token log stored verbatim in VRAM.	[PRIMARY SOURCE]
State Capacity	Finite network capacity Context capacity bounded by synaptic state and interference.	Explicit token storage Limited strictly by physical GPU VRAM.	[DERIVED]

Progression: What Changes Between Attention and State?

Transitioning from token buffers to evolving synaptic state.

01 · Transformer

02 · The Growth Problem

03 · BDH Synaptic State

04 · The Trade-off

STEP 01 · TRANSFORMER ARCHITECTURE

[PRIMARY SOURCE] Architecture

1. Transformer: Explicit Token-Addressable History

In standard Transformer attention, every historical token generates an independent Key and Value representation stored directly in GPU High-Bandwidth Memory (HBM). When attending, the model directly evaluates pairwise dot-product similarities against every past position.

Input Sequence:

VRAM Storage:

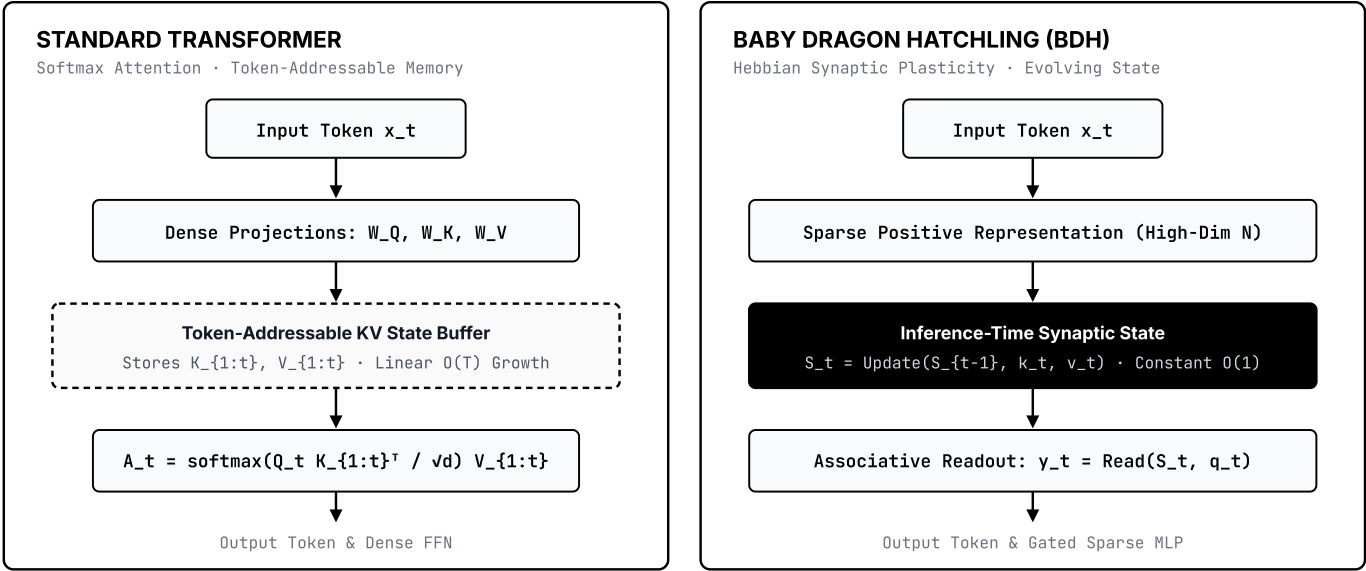
Attention:

$[x_1, x_2, \dots, x_T]$
 $[k_1, v_1], [k_2, v_2], \dots, [k_T, v_T] \rightarrow T \text{ distinct token memory vectors in VRAM}$
 $A_t = \text{softmax}(Q_t K_{\{1:t\}}^T / \sqrt{d}) V_{\{1:t\}}$

Characteristic: Every historical token preserves its exact individual position and vector representation.

Schematic: Transformer Pipeline vs. BDH Pipeline

Token-addressable buffer vs. inference-time synaptic state.



Epilogue: The Frontier of AI Memory

Whether through low-rank latent projections, selective state spaces, or synaptic plasticity, modern inference architectures seek a common objective: remembering what matters without suffocating under the weight of the past.

PRIMARY SOURCES & KEY REFERENCES

- **Kosowski et al. (2025):** *The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain*. arXiv:2509.26507.
- **Engdahl et al. (2026):** *BDH-CQ: In-Context Learning with Recurrent Latent Reasoning*. arXiv:2608.09888.
- **Kosowski (2025):** Author clarification & summary table on Hugging Face (huggingface.co/papers/2509.26507).
- **Pope et al. (2023):** *Efficiently Scaling Transformer Inference on TPU v4*. arXiv:2211.05102.
- **Vaswani et al. (2017):** *Attention Is All You Need*. NeurIPS 2017.
- **Xiao et al. (2023):** *Efficient Streaming Language Models with Attention Sinks*. arXiv:2309.17453.
- **DeepSeek-AI (2024):** *DeepSeek-V2 Technical Report* (MLA). arXiv:2405.04434.
- **Gu & Dao (2023):** *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. arXiv:2312.00752.